

PIIPS

Precision Intelligence Invoice Processing Suite — v2.1

Developer Handbook — architecture, source code, data flow and data structures, from a blank machine to a running production system.

Written for a developer with basic Python knowledge and no prior exposure to this codebase.

Table of Contents

1. Introduction & Architecture
2. Environment Setup (Zero to Running)
3. Project Folder & File Structure
4. Database Schema
5. Configuration & Secrets
6. The OCR Pipeline (ocr_engine.py)
7. Anchor-Based Field Extraction (anchor_extract.py)
8. Invoice Schema & Business Rules (invoice_schema.py)
9. Service First Integration (service_api.py)
10. Processing Orchestration (processor.py)
11. Excel Export & Field Mapping (excel_export.py)
12. Template & Format Model (template_store.py, format_model.py)
13. Database Access Layer (database.py)
14. The FastAPI Application (app.py, main.py)
15. Frontend Architecture (React)
16. Production Deployment

Chapter 1 — Introduction & Architecture

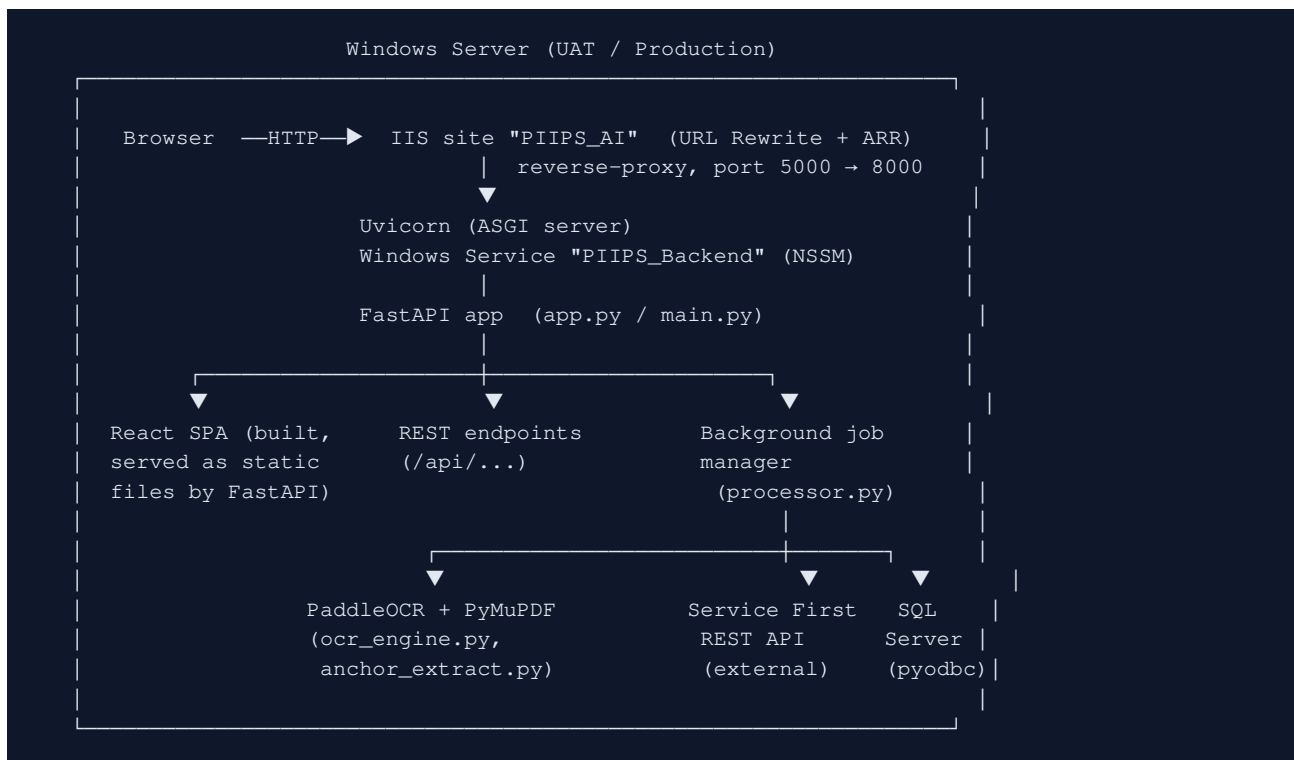
1.1 What PIIPS does

PIIPS takes a folder of vendor invoice PDFs, reads each one with OCR, extracts structured fields (seller/buyer, invoice number, line items, taxes), enriches them against an external ERP-style API ("Service First"), applies business rules, and produces:

- Rows in a SQL Server database (`tbl_Purchase_Header` / `tbl_Purchase_Line` / `tbl_Reservation_Entry`), mirroring Microsoft Dynamics 365 Business Central's Purchase Order data model.
- An Excel workbook export of the same three sheets, for manual import into Business Central.

Every invoice moves through a small state machine of statuses (see §1.3) as it's extracted, validated against Service First, and checked for missing mandatory data, ending at **READY TO LOAD** and then, once loaded into the downstream ERP by a user, **LOADED** → **POSTED** → **COMPLETED**.

1.2 System architecture

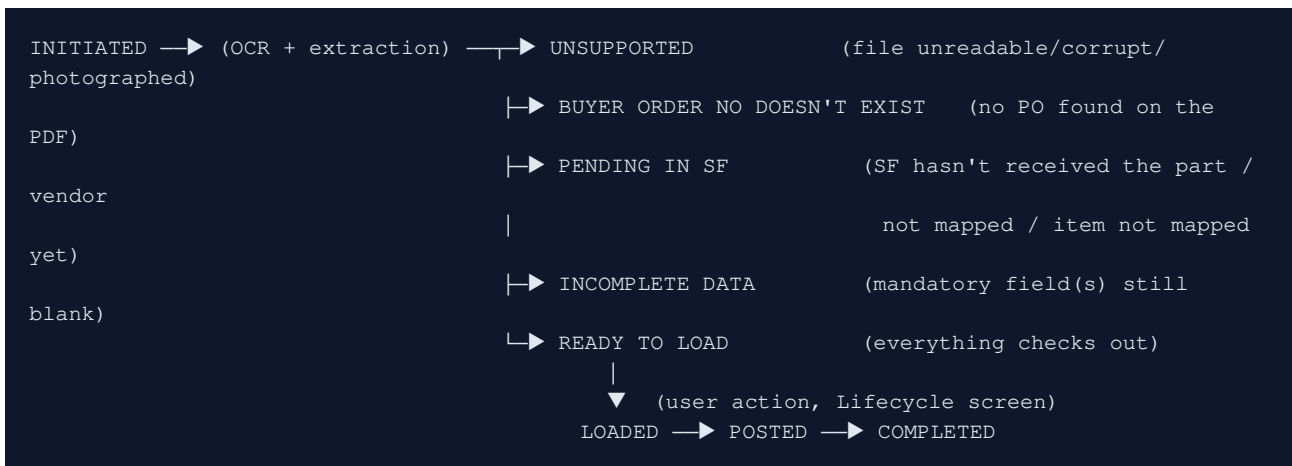


1.3 Core concepts & vocabulary

Term	Meaning
Entity	Top-level business unit a template belongs to: <code>P_I</code> , <code>P_T</code> , or <code>P_B</code> . First segment of every Input/Output/Template path.
Invoice Type	<code>PART</code> (a physical goods invoice — formerly labelled "GRN") or <code>SERVICE</code> (a services invoice, formerly "NON-GRN"). Second path segment. Drives whether Service First / Reservation Entry logic applies at all.
Template	A named business configuration (Entity + Invoice Type + Name, e.g. <code>P_T\PART\Services_Chennai</code>) holding a PO-number format and constant ("static") column values for the three output sheets.

Format	A learned OCR recognition pattern keyed by the seller's GSTIN (<code>format_model.json</code>). Decides whether an invoice's vendor layout is "known" (process normally) or "unknown" (park in <code>New_Format</code> for training).
Batch	One Dashboard "Start" run. All invoices processed in that run share a batch name, <code>PIIPS_Batch_<timestamp></code> .
Status	Where an invoice currently sits in its lifecycle (also a folder name and a <code>tbl_status</code> row). See table below.

1.4 The status lifecycle



An invoice can also be manually **EXCLUDED** from any active status (removed from Excel export without deleting its data).

1.5 Technology stack

Layer	Technology	Where
Backend web framework	FastAPI + Uvicorn (ASGI)	<code>app.py</code> , <code>main.py</code>
OCR	PaddleOCR (rasterised pages) + PyMuPDF text layer (born-digital fast path)	<code>ocr_engine.py</code>
PDF rasterisation	pdf2image (wraps Poppler)	<code>ocr_engine.py</code>
Database	SQL Server via pyodbc, ODBC Driver 17	<code>database.py</code>
Excel generation	openpyxl	<code>excel_export.py</code>
Frontend	React 18 + Vite, no router (single-page state-based navigation)	<code>frontend/</code>
Process hosting	NSSM (Windows Service wrapper around Uvicorn)	§16
Reverse proxy	IIS + URL Rewrite + Application Request Routing	§16
Secret storage	Windows DPAPI (machine-scoped)	<code>secret_store.py</code>

Chapter 2 — Environment Setup (Zero to Running)

This chapter gets a brand-new Windows machine to a running PIIPS instance on `http://localhost:8000`, for local development (production hosting under IIS is Chapter 16).

2.1 Install prerequisites

#	Install	Why
1	Python 3.10.x (64-bit), "Add to PATH" checked	Runs everything backend-side.
2	Node.js LTS	Builds the React frontend.
3	SQL Server (any edition) + SQL Server Management Studio	Application database.
4	ODBC Driver 17 for SQL Server	Python→SQL Server connectivity (pyodbc is a thin wrapper around this native driver — it does not work without it).
5	Poppler for Windows (unzip anywhere, add its <code>Library\bin</code> to the System PATH)	<code>pdf2image</code> shells out to Poppler's <code>pdftoppm.exe</code> to rasterise PDF pages into images for OCR.

2.2 Get the code and create a virtual environment

```
cd D:\WorkSpace\Projects\Python
git clone <repo-url> PIIPS      REM or copy the folder
cd PIIPS

python -m venv venv
venv\Scripts\activate
```

A **virtual environment** (venv) is a private, isolated copy of the Python interpreter and its installed packages, living entirely inside the `venv\` folder. Without it, `pip install` would install packages globally on the machine, where they can silently conflict with other projects' version requirements. Every command below assumes the venv is activated (its Scripts folder is first on PATH, so plain `python/pip` resolve to `venv\Scripts\python.exe`).

2.3 Install Python packages

```
pip install --upgrade pip
pip install -r requirements.txt
```

Key packages and what each is for:

Package	Used for
fastapi, uvicorn, starlette	Web framework / ASGI server
pyodbc	SQL Server database access
paddleocr, paddlepaddle, opencv-python	OCR engine and image processing
pdf2image	PDF page rasterisation (via Poppler)
pymupdf (import name <code>fitz</code>)	Reads the embedded text layer of born-digital PDFs (fast path, skips OCR)

openpyxl	Reads/writes the .xlsx export
pandas, numpy	Tabular/numeric helpers used across OCR row processing
requests	Calls the Service First REST API
python-docx, pdf2docx	Not used by the running app — leftover tooling dependencies, safe to ignore

2.4 Create the SQL Server database

Create an empty database (e.g. `PIIPS`) in SQL Server Management Studio. No schema script needs to be run manually — every table, stored procedure and lookup value is created automatically, idempotently, the first time the app connects (see Chapter 4 and `database.py`'s `ensure_menu_schema()`).

2.5 First run

```
cd D:\WorkSpace\Projects\Python\PIIPS
venv\Scripts\activate
python -m uvicorn main:app --reload --port 8000
```

Open `http://localhost:8000`. On first launch, before any configuration is saved, the app still serves the login page and the Configuration screens — log in (a default Super Admin is seeded the first time `init_user_table()` runs; see Chapter 13 §13.6), then:

1. **Database Configuration** — paste the SQL Server connection string. This triggers `ensure_menu_schema()`, which creates every table.
2. **Folder Configuration** — set the base folder for Input/Output/status folders (any writable local path, e.g. `D:\PIIPS_Data`). Saving it creates the folder tree automatically.
3. (Optional) **API Configuration** — the Service First base URL, if that integration is available in this environment. Without it, invoices stop at status `EXTRACTED` instead of being validated further (see Chapter 9).

2.6 Build the frontend (optional for dev)

`--reload` mode serves the frontend straight from `frontend/dist` if it already exists. To build it fresh (or after any frontend source change):

```
cd frontend
npm install
npm run build
cd ..
```

This is a static build step (Vite) — there is no separate frontend dev server / port in this project; FastAPI always serves the built files directly.

2.7 PaddleOCR's first-run download

The very first OCR call downloads PaddleOCR's recognition models (a few hundred MB) to `%UserProfile%\paddleocr`. This needs outbound internet access once; after that the models are cached locally and no further downloads happen. (Chapter 16 covers the production gotcha where this cache path differs for a Windows Service account.)

Chapter 3 — Project Folder & File Structure

3.1 Source code tree

```
PIIPS/
├─ app.py                FastAPI routes (Ch.14)
├─ main.py              ASGI entrypoint: "from app import app"
├─ processor.py         Background job orchestration (Ch.10)
├─ ocr_engine.py        PDF → OCR boxes → rows → items (Ch.6)
├─ anchor_extract.py    Header field extraction (Ch.7)
├─ invoice_schema.py    Raw OCR → business JSON (Ch.8)
├─ service_api.py       Service First integration (Ch.9)
├─ excel_export.py      JSON → Excel / DB rows, mandatory-field rules (Ch.11)
├─ template_store.py    Business template CRUD (Ch.12)
├─ format_model.py      OCR format learning/matching (Ch.12)
├─ database.py          All SQL Server access (Ch.13)
├─ config_store.py      config.json + folder helpers (Ch.5)
├─ secret_store.py      DPAPI encryption for the DB connection string (Ch.5)
├─ buyer_order.py      SPRPUR purchase-order-number detection
├─ config.json          Runtime configuration (folder path, DB, SF URL)
├─ format_model.json     Learned OCR formats (per-vendor recognition data)
├─ web.config           IIS reverse-proxy rule (Ch.16)
├─ requirements.txt     Python dependencies
├─ logo/               Static branding assets
├─ logs/               NSSM service stdout/stderr logs (production)
├─ model_backups/       Timestamped format_model.json backups (auto, before each Train)
├─ sample_input/       Sample PDFs used for testing (PART/, SERVICE/)
├─ required_softwares/  Bundled installers: NSSM, IIS URL Rewrite, ARR
├─ venv/               Python virtual environment (not committed)
├─ frontend/
│   ├─ package.json
│   ├─ vite.config.js
│   ├─ dist/           Built static site (served by FastAPI) — not committed
│   └─ src/
│       ├─ App.jsx      Shell: sidebar, routing-by-state, theming (Ch.15)
│       ├─ api.js       All fetch() calls to the backend, one per endpoint
│       ├─ Login.jsx, ForgotPassword.jsx
│       ├─ Dashboard.jsx Batches, status charts, run results
│       ├─ InputFiles.jsx File Explorer (upload / list Input)
│       ├─ BuyerOrderEntry.jsx
│       ├─ Lifecycle.jsx Load / Post / Complete (parameterised by stage)
│       ├─ Configuration.jsx, DatabaseConfig.jsx, ApiConfiguration.jsx
│       ├─ Template.jsx
│       ├─ CreateField.jsx, Mapping.jsx
│       ├─ Training.jsx
│       ├─ UserManagement.jsx
│       ├─ components.jsx Shared bits: DataTable, Modal, Logo
│       └─ index.css     Design system (CSS custom properties, themes)
```

3.2 Data folder tree (Folder Path from Configuration)

Everything under here is created and managed automatically once `folder_path` is saved — nothing here is hand-maintained.

```
<Folder Path>/
├─ Input/
│   └─ <Entity>/<PART|SERVICE>/<Template Name>/    e.g. PT/PART/Services_Chennai/
│       └─ *.pdf                                     dropped here to be processed
├─ New_Format/           Unrecognised-vendor PDFs, waiting to be trained
└─ Trained_format/       Copy of every PDF that has already been used to train a format
```

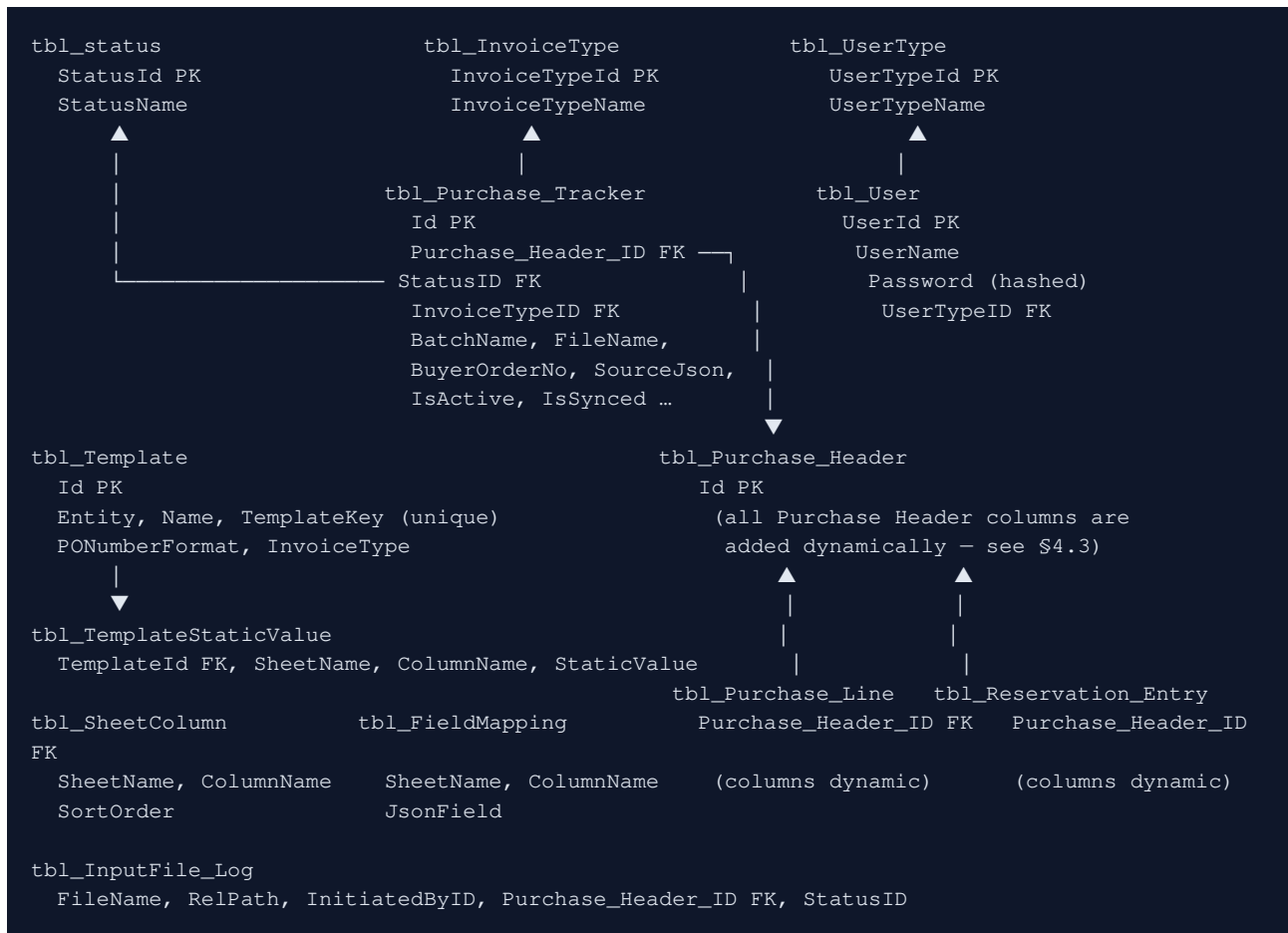
```
|— Output/
|   |— <Entity>/<PART|SERVICE>/<Template Name>/
|       |— *.json                                one extracted invoice per file
|— INITIATED/                                     (conceptual only – untouched files just sit in Input)
|— UNSUPPORTED/
|— BUYER ORDER NO DOESN'T EXIST/
|— PENDING IN SF/
|— INCOMPLETE DATA/
|— READY TO LOAD/
|— LOADED/
|— POSTED/
|— COMPLETED/
|— EXCLUDED/
```

The 9 status folders (all but Input/New_Format/Trained_format/Output) mirror `database.STATUS_VALUES` exactly — each source PDF is moved into the folder matching its current tracker status after every processing run (`config_store.move_pdf_to_status()`), so the file system always visually reflects the database.

--	--	--

Chapter 4 — Database Schema

4.1 Entity-relationship overview



4.2 Fixed-schema tables

Table	Key columns	Purpose
tbl_status	StatusId PK, StatusName (unique)	The 12 lifecycle statuses (§1.4), seeded from <code>database.STATUS_VALUES</code> .
tbl_InvoiceType	InvoiceTypeId PK, InvoiceTypeName (unique)	PART / SERVICE, seeded from <code>database.INVOICE_TYPE_VALUES</code> .
tbl_UserType	UserTypeId PK, UserTypeName	User / Admin / Accounts / Super Admin.
tbl_User	UserId PK, UserName, Password (PBKDF2-SHA256 hash), UserTypeId FK, IsActive	Login accounts.
tbl_Template	Id PK, Entity, Name, TemplateKey (unique, "Entity\InvoiceType\Name"), PONumberFormat, InvoiceType	One row per business template.
tbl_TemplateStaticValue	TemplateId FK, SheetName, ColumnName, StaticValue	Constant column values for a template, normalised one row per (sheet, column).

tbl_SheetColumn	SheetName, ColumnName, SortOrder, IsActive	Which columns exist on each of the 3 output sheets, and their order (Create Field screen).
tbl_FieldMapping	SheetName, ColumnName, JsonField	Which JSON field (or "=literal") fills each column (Field Mapping screen).
tbl_InputFile_Log	FileName, RelPath, InitiatedByID, Purchase_Header_ID FK, StatusID	Upload audit trail; resolves "who uploaded this file" for the tracker's InitiatedBy.
tbl_Purchase_Tracker	Purchase_Header_ID FK, StatusID FK, InvoiceTypeID FK, BatchName, FileName, BuyerOrderNo, SourceJson, IsActive, IsSynced, IsExcluded	One row per extracted invoice; the lifecycle/state-machine table everything else hangs off.

4.3 Dynamic-schema tables

tbl_Purchase_Header, tbl_Purchase_Line and tbl_Reservation_Entry start with only an Id/CreatedAt (and a Purchase_Header_ID FK for the two child tables). Every business column (Pay-to Name, Description, HSN/SAC Code, ...) is added at runtime by `_ensure_table()` in `database.py`, driven by whatever columns are currently configured on the Create Field screen (tbl_SheetColumn) — all as `NVARCHAR(MAX) NULL`. This is why adding a new Excel column never requires a manual `ALTER TABLE`: saving it in Create Field is enough.

```
def _ensure_table(cur, table, columns, extra=None):
    """Create the table if missing (Id + CreatedAt), then add any missing
    data columns (NVARCHAR(MAX)) plus any `extra` columns {name: ddl}."""
    cur.execute("SELECT 1 FROM sys.tables WHERE name = ?", table)
    if not cur.fetchone():
        cur.execute(
            f"CREATE TABLE {_q(table)} "
            f"(Id INT IDENTITY(1,1) PRIMARY KEY, "
            f"CreatedAt DATETIME NOT NULL DEFAULT GETDATE())"
        )
    cur.execute("SELECT name FROM sys.columns WHERE object_id = OBJECT_ID(?)", table)
    existing = {r[0].lower() for r in cur.fetchall()}
    for col in columns:
        if col.lower() not in existing:
            cur.execute(f"ALTER TABLE {_q(table)} ADD {_q(col)} NVARCHAR(MAX) NULL")
    for col, ddl in (extra or {}).items():
        if col.lower() not in existing:
            cur.execute(f"ALTER TABLE {_q(table)} ADD {_q(col)} {ddl}")
```

4.4 Schema bootstrap

Every table, seed value and stored procedure listed above is (re)created idempotently by `ensure_menu_schema()`, called on every app startup and before every save. Nothing in this schema needs a DBA to hand-run scripts — pointing a blank database at the app and saving Database Configuration is the entire setup.

Chapter 5 — Configuration & Secrets

5.1 config_store.py

A thin JSON-file store (`config.json`) holding three values: `folder_path`, `db_connection`, `sf_api_url`. Every other module reads configuration through this module, never the file directly.

```
DEFAULT_CONFIG = {
    "folder_path": "",
    "db_connection": "",
    "sf_api_url": "",
}

def load_config():
    """Merge config.json over DEFAULT_CONFIG, decrypt db_connection,
    and let web.config's appSettings override folder_path if present."""

def save_config(updates):
    """Merge `updates` in, encrypt db_connection before writing to disk."""

def folders(create=False):
    """{"input", "output", "new_format", "trained_format", "base"} absolute
    paths derived from folder_path. create=True makes them if missing."""

def status_folder(status_name, create=False):
    """/ - one of the 9 lifecycle folders."""

def move_pdf_to_status(filename, status_name, src_path=None):
    """Physically move a processed PDF into its status folder, so the file
    system always mirrors the database status."""
```

Folder path resolution has one override worth knowing: if `web.config` (IIS) has an `<appSettings>` key `folder_path`, that value wins over whatever is saved in `config.json` — this lets ops point a deployed instance at a specific data folder without touching application state.

5.2 secret_store.py

Protects the database connection string at rest using the Windows Data Protection API (DPAPI), called via `ctypes` — no third-party crypto package needed.

```
MARKER = "enc:dpapi:"

def protect(plaintext):
    """DPAPI-encrypt at LOCAL_MACHINE scope, base64, prefix with MARKER."""

def unprotect(value):
    """Reverse of protect(); returns '' if the value isn't DPAPI-protected
    (legacy plaintext) or can't be decrypted (e.g. copied from another
    machine — DPAPI keys are machine-specific)."""
```

Because DPAPI is `LOCAL_MACHINE`-scoped, a `config.json` copied to a different server has a `db_connection` value that decrypts to nothing there. This is intentional (the credential can't leak by copying the file) but means the connection string must always be re-entered fresh on a new machine (Chapter 16 covers this operationally).

5.3 Data type: the runtime config dict

Key	Type	Example
folder_path	str (absolute path)	D:\PIIPS_Data
db_connection	str (.NET-style connection string, decrypted in memory only)	Server=...;Database=PIIPS;UserId=...;Password=...;
sf_api_url	str (base URL) or ""	http://10.0.1.10:8080

--	--	--

--	--	--

--	--	--

--	--

--	--

Chapter 6 — The OCR Pipeline (ocr_engine.py)

6.1 Two extraction paths

Every page is read one of two ways, decided per-page in `OCREngine._page_inputs()`:

Path	When	How
Fast path (born-digital)	The PDF has a real embedded text layer with enough words (≥ 12 words, ≥ 60 characters)	PyMuPDF's <code>page.get_text("words")</code> reads word boxes directly — no OCR, no rasterisation, near-instant.
OCR path (scanned/photographed)	No usable text layer	The page is rasterised to an image (Poppler, 300 dpi) and run through PaddleOCR (<code>det=True, rec=True, cls=True</code>) to get word boxes with confidence scores.

Both paths converge on the exact same downstream data shape — a list of word-boxes, each `{"text", "confidence", "box", "x", "y", "left", "top", "right", "bottom", "width", "height", "center_x", "center_y"}` — so every function after this point (row grouping, anchor extraction, table parsing) is oblivious to which path produced them.

6.2 Data flow: PDF → structured items



6.3 Table column detection

Before any row is treated as an item, the table's own header row is scanned once to find each column's

approximate x-position (value_cols[c]), by matching header words against known captions:

```
elif txt in ("si", "sl", "sr"): columns["SI"] = x
elif "description" in txt: columns["Description"] = x
elif "hsn" in txt: columns["HSN"] = x
elif "qty" in txt or "quantity" in txt: columns["Quantity"] = x
elif txt == "rate" or "price" in txt: columns["Rate"] = x
elif "taxable" in txt: columns["Amount"] = x          # pre-tax base, authoritative
...
```

Every subsequent data-row token is then assigned to whichever detected column's x-position it sits closest to (`min(value_cols, key=lambda c: abs(word["x"] - value_cols[c]))`) — a position-based assignment that works regardless of how many other tokens are on the row or what order they print in, which is what lets the pipeline handle both "serial-first" and "description-first" table layouts with the same code (see §6.5).

6.4 Item-start detection

Deciding where one item's row ends and the next begins is the trickiest part of table parsing. Three signals are tried, in order, per row:

1. **Standalone serial** — the row's first token is a bare 1–3 digit number ("1", "2.") — the classic case.
2. **Serial glued to text** — a number immediately followed by letters in the same OCR token ("1CPU") — common when a scan's first column is very narrow.
3. **Serial at the table's own SI-column position, wherever it sits in the row** — some layouts print the description first and the serial two or three columns later (e.g. Laptop Panel Cover | pcs | 1 | 8473 | ...). The row is scanned for a bare small integer positioned near `value_cols["SI"]`'s x-coordinate, regardless of its position in the token list.
4. **No serial at all, but a real HSN/SAC code and real description text** — a last-resort fallback for scanned invoices missing an SI column entirely, synthesising a sequential serial number. Deliberately requires actual letters somewhere in the row (not just numbers), so a stray GST-breakdown row that slipped past table cleanup (HSN | Rate% | Taxable Amt | CGST | SGST | Total — no product name) is never mistaken for a second item.

A row that matches none of these is folded into the *previous* item's Description as a wrapped continuation line (multi-line product descriptions, serial numbers, warranty text).

6.5 Charge lines

A row whose description matches a small set of freight/courier/forwarding keywords (`CHARGE_KW`) is tagged `"charge": True` and handled slightly differently: it never needs a real HSN code to be recognised as an item (freight isn't a stock item), and downstream (Chapter 9) it's never looked up against Service First's item master — it becomes a "Charge (Item)" line in Purchase Line instead of an "Item" line.

6.6 Multi-invoice PDFs

Most PDFs contain one invoice, possibly across several pages. Some contain several distinct invoices concatenated (e.g. a batch of service-call reports saved as one file). `read_pdf()` handles this by starting a new invoice group whenever a page declares its **own** Invoice No. that differs from the current group's:

```
if page_invoice_no and groups[-1]["invoice_no"] and page_invoice_no != groups[-1]
["invoice_no"]:
    groups.append(_new_group())
```

A continuation page with no Invoice No. of its own never triggers a split, so ordinary multi-page single

invoiced are undetected duplicate print copies ("DUPLICATE FRONT ENTRY", "DUPLICATE BACK ENTRY", "DUPLICATE RECIPIENT") are detected by page text and skipped entirely so their fields/items aren't double-counted.

6.7 Photographed-page detection

A page rasterised for OCR is checked structurally for signs of being a handheld photo rather than a clean scan — a non-paper aspect ratio, a dark/non-uniform border (scene visible around the document instead of blank paper), or an uneven lighting gradient. Two of these three signals trigger rejection (`ValueError`, caught upstream and the file is moved to status **UNSUPPORTED**) — a single signal alone (e.g. a slight skew) is not enough, to avoid rejecting genuine scans.

Chapter 7 — Anchor-Based Field Extraction (anchor_extract.py)

7.1 The core idea

Rather than assuming one fixed page layout, every header field is found by its **label** ("anchor"), and its value is read either to the right of the label (same row) or, if that cell is empty/another label, below it (grid layouts). The same function handles a Tally-style "Invoice No. : ABC123" line and a grid layout with "Invoice No." / "Dated" as column headers and the real values one row below.

7.2 RIGHT_FIELDS: label → value

```
RIGHT_FIELDS = {
    "Invoice No.": ["invoice no", "invoice number", "invoice #", "inv no", "bill no"],
    "Dated": ["dated", "invoice date", "date"],
    "Buyer's Order No.": ["p.o. no", "po no", "buyer's order no", "order no", "purchase
order"],
    "Place of Supply": ["place of supply"],
    ...
}
```

For each field, every header row's text is scanned for the earliest-matching phrase. The anchor is the specific **word** whose own span contains the end of that phrase match (not "any word containing the phrase's last word" — a naive substring check would mis-anchor "Invoice No." onto "e-Way Bill No." on the same row, since both contain "no"). `_value_right_or_below()` then reads the value: scan right on the same row first (stopping before a token that is itself a recognised label, or is immediately followed by a bare ":" token — both signal "that's the next field's label, not my value"); if nothing usable is found to the right, scan the same x-column on the rows below.

7.3 Party blocks: Seller / Buyer / Consignee

The page is split at 42% of its width (`divider`); text left of the divider is scanned top-to-bottom as a state machine with one of three current parties (`section`): starts as **Seller**, switches to **Buyer** or **Consignee** when a row's left-hand text matches a section marker phrase ("bill to", "consignee", "ship to", ...), or — for a layout with no marker text of its own (e.g. an informal "To," letter greeting) — an exact whole-line match against a shorter marker list. Within a section, the first non-label line becomes that party's Name; every subsequent line is appended to its Address.

A structural fallback exists for the case where even that marker text is missing from the page's text layer entirely (rendered as a graphic, not real text, on some invoice-generator tools): if a row's right-hand column (past the divider) duplicates its left-hand text exactly, that's recognised as the start of a mirrored two-column Bill-to/Ship-to block, and the section switches without needing a marker phrase at all.

7.4 Skip rules

A growing list of structural and keyword-based skip rules prevents non-address content from polluting Name/Address fields, added reactively as real invoices exposed each case: bare phone-number lines, website URLs, hex-hash fragments (wrapped IRN halves), the GST e-Invoice QR block (IRN/Ack No./Ack Date), and known non-address metadata captions ("Reverse Charge", "Credit Days").

7.5 Output shape

```
extract(header_rows, footer_rows, page_width) -> dict, e.g.:
{
  "Invoice No.": "GIRI/26-27/0429", "Dated": "17-07-2026",
  "Seller Name": "...", "Seller Address": "...", "Seller GSTIN/UIN": "...",
  "Buyer Name": "...", "Buyer Address": "...", "Buyer GSTIN/UIN": "...",
  "Buyer's Order No.": "...", "Place of Supply": "...",
  "Amount Chargeable (in words)": "...",
}
```

This dict becomes `Fields` on the OCR result; Chapter 8 turns it (plus the `Items` list from Chapter 6) into the full business JSON.

Chapter 8 — Invoice Schema & Business Rules

(invoice_schema.py)

8.1 build_invoice_json()

Converts one OCR result (`Fields` dict + `Items` list, Chapter 6–7's output) into the flat business dict every other module works with. This is the single most important data structure in the system — it is the JSON written to `Output/<entity>/<type>/<name>/<file>.json`, the shape `service_api.py` enriches in place, and the source every Excel/DB column is ultimately resolved from.

8.2 The invoice dict shape

Key	Type	Source
invoice_no, invoice_date	str	PDF (anchor-extracted)
seller_name, seller_gstin, seller_address, seller_state	str	PDF
buyer_name, buyer_gstin, buyer_address, buyer_state	str	PDF
ship_to_name, ship_to_address	str	PDF Consignee block, falls back to Buyer if absent
buyer_order_no, buyer_order_doubtful	str, bool	PDF (via <code>buyer_order.py</code> 's SPRPUR pattern detection)
GST_Vendor_Type	"Registered" / "UnRegistered"	Derived , not extracted: "Registered" whenever <code>seller_gstin</code> is non-blank.
payment_terms_code, Due_Date	str (days), str (date)	Computed — §8.3
items	list[dict]	from <code>Items</code> (Chapter 6), one dict per line — see §8.4
sf_items	list[dict]	Populated later by <code>service_api.py</code> (Chapter 9) — empty until then
_static	dict	Attached later by <code>processor.py</code> — the matched Template's constant values, kept out of the written JSON file

8.3 Payment Terms / Due Date rule

Exact precedence, driven by Service First's `PaymentTermsName` field (see Chapter 9) with a PDF and default fallback:

```
def payment_terms_days(terms_text):
    # "100% ADVANCE PAYMENT" -> 0
    # "45 DAYS" / "15 DAYS" / "7 DAYS" / ... -> that integer
    # "AGAINST INVOICE" -> None (caller falls back to the PDF's own value)
    # anything else / blank -> None

def resolve_payment_terms(sf_terms, pdf_terms):
    # 1. try payment_terms_days(sf_terms)           - Service First wins
    # 2. else try payment_terms_days(pdf_terms)      - PDF's own value
    # 3. else DEFAULT_PAYMENT_TERMS_DAYS = 45       - hard default
    # returns (terms_code: str, terms_days: int)
```

```
def add_days_to_date(date_str, days):  
    # Due_Date = Document Date + terms_days
```

8.4 The per-line item dict

Key	Meaning
Description, Quantity, Rate, Amount, HSN (as <code>hsn</code>)	Straight from the OCR item (Chapter 6)
Type	"Charge (Item)" if the OCR item was tagged a charge line, else "Item"
ProductNo	Seeded from the PDF's own HSN/SAC column value; overwritten by Service First's Nav Item No. when it resolves one (Chapter 9) — the PDF value only survives when SF finds nothing
Nav_Item_No	"" until Service First resolves it
Line_No	<code>serial * 10000</code> — Business Central's line-numbering convention (10000, 20000, 30000, ...)
PurchaseOrderNo	the invoice's <code>buyer_order_no</code> , lower-cased (join key for the Service First HSN lookup)

Chapter 9 — Service First Integration (service_api.py)

9.1 Two external calls

Endpoint	Keyed by	Returns
POST /api/purchase/ GetSparePurchaseItem	Buyer's Order No. (the PO)	One row per part physically received against that PO — becomes <code>sf_items</code> , the raw material for Reservation Entry rows. Carries <code>Nav_VendorCode</code> , <code>Nav_Item_No</code> , <code>PaymentTermsName</code> , <code>Location_Code</code> , etc.
POST /api/Purchase/ GetHSNDetails	(PurchaseOrderNo, PartSpecification=Description)	Item-master detail for one line: <code>Nav_Item_No</code> , <code>ProductNo</code> , <code>HSN_Type</code> , <code>TaxPercentage</code> , <code>Nav_Part_Description</code> .

Both are best-effort: a network error, empty `sf_api_url`, or non-2xx response degrades to an empty result rather than raising — PIIPS keeps working (at a lower validation status) when Service First is unreachable.

9.2 `enrich_invoice()` data flow

```
enrich_invoice(data)
├─ order_no = data["buyer_order_no"]
├─ rows = GetSparePurchaseItem([order_no])      → data["sf_items"] = rows
├─ header fields copied from rows[0]            (Nav_VendorCode, Location_Code, ...)
├─ _enrich_items(data, order_no)
│   for each non-charge item:
│       GetHSNDetails([PartSpecification: item.Description, PurchaseOrderNo: order_no])
│       → item["Nav_Item_No"], item["ProductNo"] (only overwritten if SF has one),
│         item["Nav_Part_Description"], item["TaxPercentage"], ...
│       → item["_hsn_nav_item_no_missing"] = True when GetHSNDetails found
│         no match / a match with a blank Nav_Item_No — SF's item master
│         genuinely doesn't recognise this description, independent of
│         any later fallback below
├─ Source Ref. No. linking: match each sf_items row to a Purchase Line
│   item by Nav_Item_No (preferred), falling back to positional
│   pairing (same sequence on the PO, quantity-expanded) when the
│   HSN lookup resolved nothing for either side
└─ return evaluate_invoice(data)
```

9.3 `evaluate_invoice()` — the verdict chain

Checks run in order; the first failure wins. Each returns `{"status", "is_active", "is_synced", "errors", "reason"}`.

#	Check	Status if failed
1	Seller GSTIN not captured from the PDF	PENDING IN SF
2	Buyer's Order No. empty	BUYER ORDER NO DOESN'T EXIST
3	Buyer's Order No. format looks doubtful (garbled OCR)	BUYER ORDER NO DOESN'T EXIST

4	(only when <code>sf_api_url</code> is configured) No <code>sf_items</code> at all — part not yet received	PENDING IN SF
5	Any non-charge item's <code>_hsn_nav_item_no_missing</code> is True — Service First's item master doesn't recognise this part's description at all	INCOMPLETE DATA, flagged for retraining (<code>new_format: True</code> — Chapter 10 copies the source PDF into <code>New_Format</code>)
6	Any non-charge item still has no Nav Item No./ProductNo after all enrichment	PENDING IN SF
7	Any <code>sf_items</code> row itself has a blank <code>Nav_Item_No</code>	PENDING IN SF ("Nav item No not mapped in SF")
8	Any <code>sf_items</code> row itself has a blank <code>Nav_VendorCode</code>	PENDING IN SF ("NAV Vendor Code not Mapped in SF")
9	All of the above pass	READY TO LOAD, <code>is_synced: True</code>

This verdict is **provisional**: `processor.py`'s final mandatory-field gate (Chapter 10 §10.4) can still re-classify a READY TO LOAD invoice down to INCOMPLETE DATA, or promote a PENDING IN SF one up to READY TO LOAD, based on whether every mandatory column actually ended up filled — the field-completeness check is the real, final authority on status.

--	--	--

--	--	--

--	--	--

Chapter 10 — Processing Orchestration (processor.py)

10.1 Job manager

A `JobManager` singleton runs each Dashboard "Start" click as a background thread (`Job`), tracked by a short job id, polled by the frontend via `/api/process/status/<job_id>`. Only one job runs at a time; a second Start while one is active gets HTTP 409.

10.2 Per-file pipeline (mode="process")

```
for each PDF in Input (recursively):
    ocr_result = OCREngine().read_pdf(src_path)           Chapter 6
    for each invoice_ocr in ocr_result["Invoices"]:       (usually 1)
        data = build_invoice_json(invoice_ocr, src_path)  Chapter 8
        data["Template_Name"], data["Template"] = fmt_name
        data["GST_Vendor_Type"] = derived from seller_gstin
        if no invoice_no extracted: → moved to New_Format, skip
        if invoice_no already processed this run: → skip (dedup)
        static, template_key = template_store.static_for_path(...)
        invoice_type = template_store.invoice_type_for_path(...) PART / SERVICE
        doc_type, doc_no = derived from static + a per-job sequence counter
        data["Document Type"], data["Document No."] = doc_type, doc_no
        if invoice_type == "SERVICE":
            verdict = {"status": "READY TO LOAD", ...}    - no SF, no reservations
        else:
            verdict = service_api.enrich_invoice(data)    Chapter 9
        if verdict["new_format"]:
            copy source PDF into New_Format (for retraining) - invoice still saves
            resolve Payment Terms Code / Due Date        $8.3
            stamp Document Type/No., Source Type=39, Source Ref. No. onto every item
            write data as Output/<entity>/<type>/<name>/<file>.json
            record = {file, status, data, _status, _isactive, _synced, _invoice_type}
```

10.3 fmt_model.match() gate

Before any of the above runs, the file's OCR result is matched against known formats by seller GSTIN (Chapter 12). No match → the file is moved straight to `New_Format` and mirrored to a training-server copy if configured; none of the extraction/validation pipeline above runs for it at all.

10.4 _save_to_db(): the final mandatory-field gate

After every file in the run has been extracted, one bulk save happens. Before that save, every invoice (except ones already at BUYER ORDER NO DOESN'T EXIST — that status has its own manual-entry workflow) is re-checked against the mandatory Header/Line/Reservation field list (Chapter 11 §11.3), **regardless of what `evaluate_invoice()` said:**

```
for i, group in enumerate(grouped["groups"]):
    if tracker["statuses"][i] == "BUYER ORDER NO DOESN'T EXIST":
        continue
    missing = excel_export.missing_required_fields(header, lines, reservations)
    if missing:
        tracker["statuses"][i] = "INCOMPLETE DATA"
        if not matched[i].get("reason"): # keep a more specific SF reason if one exists
            matched[i]["reason"] = "Missing required field(s): " + ", ".join(missing)
    else:
```

```
tracker["statuses"][i] = "READY TO LOAD"
```

This is why the status a file lands on can differ from what Chapter 9's verdict chain alone would suggest: field completeness is checked last and wins.

10.5 File organisation after save

`_move_by_status()` moves every processed source PDF into the status folder matching its **final** tracker status (not the pre-gate verdict) — `matched[i][ "_status" ]` is kept in sync with `tracker["statuses"][i]` in §10.4 specifically so this step files the PDF in the folder the database agrees with.

10.6 Other job modes

Mode	Reads from	Effect
train	New_Format	Runs OCR + <code>fmt_model.learn()</code> per file (Chapter 12), no extraction/save.

Chapter 11 — Excel Export & Field Mapping

(excel_export.py)

11.1 Column value resolution order

For every (sheet, column) cell, in this exact precedence:

```
_row(inv, item, sheet, smap, cols):
    1. _link_override() - hardcoded relationship keys (Document Type, No./
                        Document No., Line No., Source Type/Subtype/ID/
                        Ref. No., Reservation Quantity=1) - always wins
    2. mapped JSON field - tbl_FieldMapping's configured source field
    3. template static   - tbl_TemplateStaticValue, when no mapping exists
```

11.2 Field source classification

The Dashboard's "Fields" drill-down (Super Admin only) labels each field's origin using this same logic, one layer up:

Source shown	When
Service First	Every Reservation Entry column (the table doesn't exist without an SF reservation row); any Purchase Header/Line column mapped to a known SF-populated JSON field (Location_Code, Nav_Item_No, PaymentTermsName, ...).
Template	An unmapped column (falls back to the static value), or a hardcoded relationship key like Document Type.
PDF	Everything else — a column mapped to a field the OCR/anchor pipeline extracted directly.

11.3 Mandatory fields

```
REQUIRED_HEADER_FIELDS = [
    "Pay-to Name", "Pay-to Address", "Ship-to Name", "Ship-to Address",
    "Posting Date", "Payment Terms Code", "Due Date", "Location Code",
    "Vendor Posting Group", "Vendor Invoice No.", "Buy-from Vendor Name",
    "Buy-from Address", "Ship-to Country/Region Code", "Bal. Account Type",
    "Status", "IC Status", "State", "Structure", "GST Vendor Type",
    "GST Order Address State", "Is Employee", "InvoiceNo",
]
REQUIRED_LINE_FIELDS = [
    "Line No.", "Type", "Location Code", "Description",
    "Quantity", "Direct Unit Cost", "Line Amount", "TDS Nature of Deduction",
]
# + conditionally, by the line's own Type:
_LINE_TYPE_REQUIRED_FIELD = {"Item": "No."} # Charge (Item) lines: no extra requirement
REQUIRED_RESERVATION_FIELDS = [
    "Positive", "Item No.", "Location Code", "Quantity (Base)",
    "Reservation Status", "Creation Date", "Source Type",
    "Expected Receipt Date", "Serial No.", "Qty. per Unit of Measure",
    "Quantity", "Suppressed Action Msg.", "Planning Flexibility",
    "Qty. to Handle (Base)", "Qty. to Invoice (Base)",
    "Disallow Cancellation", "Correction", "Item Tracking", "Untracked Surplus",
]
```


Three columns are deliberately omitted: Header No., Line Document No., Reservation Source ID (all three are the same document number, only made real at Load time) and Reservation Source Ref. No. (only resolved once Document No./Line No. become real). Requiring them early would wrongly downgrade an otherwise-complete invoice.

11.4 Document No. / Entry No. renumbering (export-time only)

`renumber_batch()` lets the Dashboard's Batches table override the auto-generated Document No. and Entry No. sequence **at download time**, without touching the database: given a starting number, each header's No. is reassigned sequentially (keeping its existing prefix/zero-padding), the same new value is propagated onto every Purchase Line's Document No. and Reservation Entry's Source ID that referenced the old one, and Reservation Entry rows get a fresh sequential Entry No. across the whole batch.

--	--	--

--	--	--

--	--

--	--	--

--	--	--

--	--

--	--

--	--	--

Chapter 12 — Template & Format Model

12.1 Template vs. Format — two decoupled systems

	Template (template_store.py)	Format (format_model.py)
Keyed by	Entity + Invoice Type + Name (a path)	Seller GSTIN
Decides	Which folder a PDF sits in; constant column values (Document Type, Location Code, ...)	Whether the vendor's layout is "known" at all — routes unknowns to New_Format
Stored in	tbl_Template / tbl_TemplateStaticValue	format_model.json (flat file, not the database)
Changed via	Template screen	Model Training screen (Train button)

They are intentionally independent: a template decides *where a file goes and what constants apply*; a format decides *whether PIIPS recognises the invoice at all*. A PDF can be correctly filed under a template yet still be an "unknown format" if this specific vendor's GSTIN has never been trained.

12.2 Template folder convention

```
save_template(entity, invoice_type, name, po_format, static, user_id=None)
→ creates <Input>/<entity>/<invoice_type>/<name>/
→ key = f"{entity}\\{invoice_type}\\{name}"      e.g. "PT\\PART\\Services_Chennai"

static_for_path(input_folder, pdf_path)
→ parses (entity, invoice_type, name) back out of a PDF's own path
→ looks up that key's static values
```

12.3 format_model.py

```
learn(ocr_result, filename, persist=False)
→ keys a new/updated format entry by the invoice's seller GSTIN
→ persist=False during a training RUN (many files); the model is written
  once at the end, atomically, so a failed run leaves it untouched

match(ocr_result)
→ looks up the extracted seller GSTIN against known formats
→ returns (format_name, score) or (None, 0) if unrecognised
```

A backup of `format_model.json` (`model_backups/`) is taken automatically before every Train run, so a bad training pass can be rolled back by restoring the previous file.

Chapter 13 — Database Access Layer (database.py)

13.1 Connection

`get_connection()` converts the .NET-style connection string from `config_store.load_config()` into a pyodbc connection string (ODBC Driver 17), inferring Windows vs. SQL auth from whether a `User Id` is present.

13.2 Schema bootstrap: `ensure_menu_schema()`

Runs the entire `_MENU_TABLE_DDL` list (every `CREATE TABLE IF NOT EXISTS`, every `ALTER TABLE ADD COLUMN IF NOT EXISTS`, every seed `INSERT`) plus `_MENU_TYPE_DDL` (table-valued parameter types) and `_MENU_PROC_DDL` (stored procedures, all `CREATE OR ALTER`). Called on every app startup and before every save-path function — safe to call as often as needed; every statement is idempotent.

13.3 Bulk save via stored procedure + JSON

Rows are never inserted one at a time in a Python loop. `save_grouped()` serialises every header/line/reservation row (plus the tracker metadata) to JSON and hands it in one call to `usp_SaveInvoiceBatch`, which uses `OPENJSON` + a temp table (`#hmap`, group-id → new header Id) to resolve foreign keys and insert everything in a handful of set-based statements.

```
EXEC dbo.usp_SaveInvoiceBatch
    @HeaderCols='["InvoiceNo",...]', @LineCols=[...], @ResCols=[...],
    @Headers='[{"_gid":0,"_StatusName":"READY TO LOAD",...}, ...]',
    @Lines=[...], @Reservations=[...],
    @StartedByID=7, @StartedDatetime='...', @StatusName='EXTRACTED'
```

Column names are escaped for `OPENJSON` with `REPLACE(value, '"', '"\ "')`, not SQL Server's `STRING_ESCAPE(..., 'json')` — the latter also escapes `/` to `\`, which `OPENJSON`'s path parser doesn't un-escape, silently nulling any column with a slash in its name (e.g. "HSN/SAC Code"). This was a real bug found and fixed during development — worth knowing if a similar escaping helper is ever reused elsewhere.

13.4 Dynamic table columns

See Chapter 4 §4.3 — `_ensure_table()` adds any Purchase Header/Line/Reservation Entry column that `Create Field (tbl_SheetColumn)` currently defines but the physical table doesn't yet have, as `NVARCHAR (MAX)` `NULL`, every time `save_grouped()` runs.

13.5 Key read functions

Function	Returns	Used by
<code>fetch_batch(batch_name, sheet_cols)</code>	{sheet: {columns, rows}} for one batch's active, non-excluded invoices	Excel download
<code>invoices_by_status(status_id),</code> <code>invoices_by_batch(batch)</code>	Flat invoice list for a Dashboard pop-up	Dashboard drill-downs

<code>get_invoice_field_check(header_id)</code>	Every mandatory field, its value, whether it's missing, and its source	Super Admin "Fields" pop-up
<code>resync_pending()</code>	Re-attempts Service First for every PENDING IN SF record, promotes what's now available	Called at the start of every processing run
<code>advance_status(header_ids, from_statuses, to_status)</code>	Bulk status transition	Load / Post / Complete screens

13.6 Users & roles

```
USER_TYPE_VALUES = ["User", "Admin", "Super Admin", "Accounts"]

hash_password(password)      # PBKDF2-SHA256, salted - one-way, never reversed
authenticate(username, password)
create_user(username, password, user_type_id, created_by=None)
reset_password(username, new_password, modified_by=None)
get_username_and_role(user_id) # {"username", "role"} - used to stamp
                                # downloaded manual PDFs (Chapter 15 $15.4)
```

Chapter 14 — The FastAPI Application (app.py, main.py)

14.1 Entry point

```
# main.py — the whole file
from app import app
```

Uvicorn is pointed at `main:app`. `main.py` exists purely so the ASGI target name is short and stable (Chapter 16's NSSM/Uvicorn command line references it directly) even as `app.py` itself grows.

14.2 Route groups

Prefix	Covers
<code>/api/config</code> , <code>/api/api-config</code>	Folder Path / DB connection / Service First URL (Chapter 5)
<code>/api/process/*</code>	Start a job, poll status, fetch results (Chapter 10's <code>JobManager</code>)
<code>/api/train</code> , <code>/api/train/files</code>	Model Training screen (Chapter 12)
<code>/api/templates</code>	Template CRUD (Chapter 12)
<code>/api/columns</code> , <code>/api/mapping</code>	Create Field / Field Mapping screens (Chapter 11)
<code>/api/stats/status-counts</code> , <code>/api/invoices/by-status</code> , <code>/api/invoices/by-batch</code> , <code>/api/invoices/<id>/fields</code>	Dashboard charts and drill-downs
<code>/api/lifecycle/*</code>	Load / Post / Complete (Chapter 13 §13.5)
<code>/api/batches</code> , <code>/api/batches/download</code>	Excel export (Chapter 11)
<code>/api/manual/download</code>	Role-gated manual PDF download, stamped with the requester's identity — Chapter 15 §15.4
<code>/api/users/*</code>	User Management screen (Chapter 13 §13.6)

14.3 Startup hook

```
@app.on_event("startup")
def _bootstrap_menu_storage():
    config_store.ensure_secret_encrypted() # migrate any plaintext DB secret
    if config_store.load_config().get("db_connection"):
        database.migrate_menu_json(BASE_DIR) # ensure_menu_schema() + legacy JSON import
        database.migrate_template_invoice_type() # one-time backfill, now a no-op in practice
```

Wrapped in a broad `try/except` — a DB that isn't configured or isn't reachable yet must never prevent the server itself from starting (the Configuration screens have to be reachable to fix that).

14.4 Serving the frontend

The built React app (`frontend/dist`) is mounted as static files at the root; every unmatched non-`/api` path falls through to `index.html`, letting the React app's own state-based navigation (no client-side router, no

jQuery handles everything after the initial page load.

jQuery handles everything after the initial page load.

jQuery handles everything after the initial page load.

jQuery handles everything after the initial page load.

jQuery handles everything after the initial page load.

jQuery handles everything after the initial page load.

Chapter 15 — Frontend Architecture (React)

15.1 No router — state-based navigation

The app is a single Vite/React SPA with **no react-router**. `App.jsx` holds the current screen in a plain `useState("dashboard");` the sidebar's `onClick` calls `setPage(key)` directly. There is exactly one URL for the whole application after login — screens are never deep-linkable.

```
const MENU = [
  ["dashboard", "Dashboard", "🏠", "Main"],
  ["input", "File Explorer", "📁", "Main"],
  ["manual", "Manual", "📖", "Main"],
  ["buyerorder", "Buyer Order Entry", "👤", "Review"],
  ["load", "Load", "👤", "Accounts"], ["post", "Post", "👤", "Accounts"],
  ["complete", "Complete", "✓", "Accounts"],
  ["configuration", "Folder Configuration", "⚙️", "Setup"], ...
];

const ROLE_MENUS = {
  "super admin": MENU.map(([k]) => k), // everything
  admin: ["dashboard", "input", "manual", "buyerorder", "load", "post", "complete",
    "configuration", "apiconfig", "template", "createfield", "users"],
  user: ["dashboard", "input", "manual", "buyerorder", "load"],
  accounts: ["dashboard", "input", "manual", "post", "complete"],
};

const PAGES = { dashboard: Dashboard, input: InputFiles, manual: Manuals, ... };
// render:   where Active = PAGES[activePage]
```

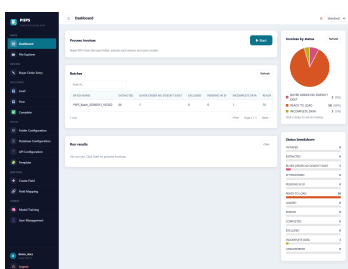
Every page component receives the same `user` prop (from `localStorage.piips_user`, set once at login) and is free to render role-specific UI itself — e.g. `Manuals.jsx` only shows the Developer Manual download card when `user.user_type` is Super Admin.

15.2 api.js

Every backend call lives in one file as a plain exported function returning a `fetch()` promise — no state management library, no generated client. Screens call these directly and manage their own local `useState` for results.

```
export const getBatches = () => request("/api/batches");
export const batchDownloadUrl = (batch, docNo, entryNo) => `${API_BASE}/api/batches/download?...`;
export const manualDownloadUrl = (userId, kind="user") => `${API_BASE}/api/manual/download?user_id=${userId}&kind=${kind}`;
```

15.3 Screens, at a glance



The Dashboard: batch table, live status pie chart, and per-status breakdown — every count is clickable and opens an invoice-list pop-up (DataTable + Modal, both in `components.jsx`, reused across every screen in the app).

15.4 The Manual download feature

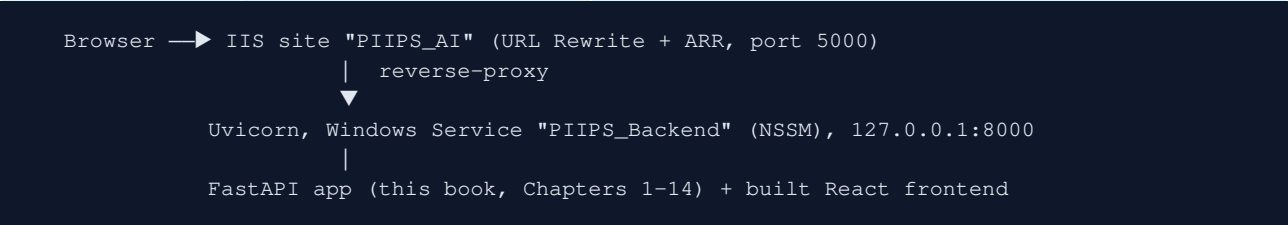
`Manuals.jsx` is the newest screen — a role-aware download page rather than a data screen. It calls the same `manualDownloadUrl()` helper as every other download link in the app (batches use the identical `<a href=...>` pattern), but the backend endpoint behind it does something the others don't: it re-derives the caller's role **server-side** from `user_id` (never trusting a client-supplied role) and stamps every page of the returned PDF with who downloaded it and when, using PyMuPDF:

```
# app.py
@app.get("/api/manual/download")
def download_manual(user_id: int, kind: str = "user"):
    who = database.get_username_and_role(user_id) # {"username", "role"}
    if kind == "developer" and who["role"].lower() not in ("super admin", "developer"):
        raise HTTPException(403, "The Developer Manual is Super Admin only")
    doc = fitz.open(src_path) # the pre-built role PDF
    stamp = f"Downloaded by {who['username']} ({who['role']}) on {datetime.now():%Y-%m-%d %H:%M}"
    for page in doc:
        page.insert_text(fitz.Point(28, page.rect.height - 18), stamp, fontsize=7.5)
    doc.save(out_path, garbage=4, deflate=True) # compress — an unstamped save() can
    bloat 50x
    return FileResponse(out_path, ...)
```

The 4 role-specific User Manual PDFs (and the Developer Manual) are pre-built static files in `manuals/` — built by a small standalone script that renders the same screen-content dict into 4 different HTML documents (one per role's menu list) and feeds each through the same PyMuPDF `story` HTML-to-PDF pipeline used to build this book. Regenerating them after a UI change is a matter of re-running that script and restarting the service — see the note in Chapter 16 about keeping manuals in sync.

Chapter 16 — Production Deployment

Full step-by-step instructions (prerequisites, NSSM service setup, IIS reverse-proxy configuration, the DPAPI/PaddleOCR-cache gotchas, verification checklist, troubleshooting table) live in the companion document **PIIPS_UAT_Deployment_Guide.pdf**, in the project root next to this book. Summary of the shape of it:



Gotcha	Why	Where covered
DPAPI-encrypted <code>db_connection</code> doesn't survive a copy to a new machine	LOCAL_MACHINE-scoped encryption key (Chapter 5 §5.2)	Deployment Guide §6
PaddleOCR's model cache is empty under a fresh NSSM service (Local System account)	Models download to the <i>current user's</i> profile; Local System's profile is a different, usually-empty one	Deployment Guide §8
Poppler must be on the System PATH, not just the interactive user's	The service runs under a different account than whoever set up the machine interactively	Deployment Guide §2, §11
Port must not collide with another site already on the server	Only one process can bind a given port	Deployment Guide §9.1

--	--	--

--	--	--

--	--	--

--	--	--

--	--	--